# CI DIGITAL

MPPT

# PLANO DE VERIFICAÇÃO

**Grupo:**

- ENZO
- GABRIEL MARCARINI CARAMURU
- LUCAS

**Data:** 28 de Outubro de 2026

**Rev.: 1.0**

1. ***Document Versions***

| Número da Versão | Data | Autor | Sumário de Mudanças |
|---|---|---|---|
| 1.0 | 27/10/2026 | Gabriel Marcarini Caramuru | Criação do arquivo |

Obs.: Os planos de verificação podem ser observados neste arquivo, entretanto é melhor visualizá-los separadamente nas pastas do repositório.

# SPI Controller Verification Test Plan

## 1. Overview and Scope

This document details the verification plan for the `spi_controller` RTL module. The Design Under Test (DUT) is a parameterized SPI Master module, intended as a communication submodule for an MPPT system.

The verification strategy focuses on RTL-level simulation to ensure functional correctness, data integrity, and strict adherence to protocol timing constraints. This includes validating all four standard SPI modes, customizable clock generation, and handshake signals.

## 2. Test Environment Strategy

The primary testbench strategy relies on a **Self-Checking Loopback** topology. By connecting the `SDO` (Serial Data Out) directly to the `SDI` (Serial Data In), the testbench can automate data integrity checks by comparing the transmitted parallel word (`SDO_DATA`) with the received parallel word (`SDI_DATA`).

Protocol timing checks will be validated using waveform analysis and procedural assertions within the testbench to prepare the design for accurate static timing analysis (STA) and gate-level synthesis.

## 3. Test Cases

## 3.1 Basic Initialization and Reset (Smoke Test)

**Objective:** Verify the module safely recovers from reset and maintains safe idle states.

- **Stimulus:** Assert `RST_N` (active low) asynchronously during active system clock (`CLK`).
- **Expected Result:**
  - All internal state machines immediately return to `ST_IDLE`.
  - Outputs `BUSY`, `DONE`, `SCLK`, and `SDO` drop to 0 (or `CPOL` equivalent for `SCLK`).
  - Chip Select `CS_N` drives high (inactive) within 1 system clock cycle.
  - Inputs are ignored while `RST_N` is low.

## 3.2 Clock Divider Verification (`CLK_DIV`)

**Objective:** Validate that the SPI serial clock frequency is correctly derived from the system clock based on the `CLK_DIV` parameter.

- **Stimulus:** Parameterize the DUT with `CLK_DIV` = 1, 2, and 5. Initiate a dummy transfer.
- **Expected Result:**
  - The measured period of `SCLK` exactly matches the formula: `T_sclk = 2 * CLK_DIV * T_clk`.
  - For `CLK_DIV = 1`, `SCLK` runs at half the system clock frequency.

## 3.3 SPI Protocol Modes Validation (`CPOL` & `CPHA`)

**Objective:** Ensure the module correctly implements all four standard SPI communication modes.

- **Stimulus:** Run sequential transfers with configurations:
  - Mode 0: `CPOL=0, CPHA=0`
  - Mode 1: `CPOL=0, CPHA=1`
  - Mode 2: `CPOL=1, CPHA=0`

    - Mode 3: `CPOL=1, CPHA=1`
- **Expected Result:**
    - **Idle State:** `SCLK` matches the value of `CPOL` when `BUSY` is low.
    - **Shift/Sample Edges:** Data on `SDO` shifts on the correct propagating edge, and `SDI` is sampled on the correct sampling edge, precisely determined by the `CPHA` setting.

## 3.4 Data Integrity and Loopback (Full-Duplex)

**Objective:** Verify that data parallel-to-serial and serial-to-parallel conversions occur flawlessly (MSB first).

- **Stimulus:**
    - Connect `SDO` to `SDI` in the testbench.
    - Parameterize `DATA_WIDTH` = 12.
    - Send varied patterns: `12ThA5A` (alternating), `12ThFFF` (all ones), `12Th000` (all zeros), `12Th001` (walking one).
- **Expected Result:**
    - For every initiated transfer, upon the assertion of the `DONE` signal, `SDI_DATA` must exactly equal `SDO_DATA`.
    - The first bit transmitted on `SDO` must be the MSB of `SDO_DATA`.

## 3.5 Handshake & Protocol Timing Constraints

**Objective:** Validate structural protocol timing, ensuring safety margins for external SPI peripherals.

- **Stimulus:** Initiate a standard 12-bit transfer and monitor the transaction boundaries.
- **Expected Result:**
    - **CS Setup:** `CS_N` must assert (go low) at least 1 full `SCLK` period before the first active edge of `SCLK`.
    - **CS Hold:** `CS_N` must remain asserted (low) for at least 1 full `SCLK` period after the final active edge of `SCLK`.
    - **Done Pulse:** `DONE` must pulse high for exactly 1 system clock (`CLK`) cycle upon completion of

`ST_IDLE_GAP`.

- **Inter-Transaction Gap:** If a new `START` is immediately requested, the module must ensure `CS_N` remains de-asserted (high) for a minimum of 2 `SCLK` periods before starting the next transaction.

# Sample Formatter Verification Test Plan

## 1. Overview and Scope

This document details the verification plan for the `sample_formatter` RTL module. The Design Under Test (DUT) receives 16-bit SPI frames containing a 4-bit Channel ID and a 12-bit ADC sample. The module extracts both fields, zero-extends the ADC sample to 16 bits, routes the formatted sample to the appropriate output according to the Channel ID, and generates a one-clock-cycle `sample_valid` pulse indicating that a new formatted sample is available.

**Input Signals:** `clk` , `rst_n` , `spi_frame_rx` , `spi_data_valid`
**Output Signals:** `v_sample_out` , `i_sample_out` , `sample_valid`

## 2. Verification Strategy

The verification strategy is based on RTL simulation using a self-checking testbench combined with SystemVerilog Assertions (SVA). Functional correctness is verified by comparing the DUT outputs against a reference formatter model implemented in the testbench. Assertions are used to verify FSM behavior, reset behavior, output protocol, and timing of the `sample_valid` signal.

# 3. Test Cases

## 3.1 Reset and Inicialization

**objective:** Ensure that reset initializes correctly.

- **Stimulus:** Assert and de-assert `rst_n` (active-low).
- **Expected Result:**
- FSM returs to `STATE_IDLE` ;
- `v_sample_out = 16'h0000` ;
- `i_sample_out = 16'h0000` ;
- `sample_valid = 1'b0` ;
- `r_formatted_data = 16'h0000` ;
- `r_channel_id = 4'h0` ;
- `r_adc_sample = 12'h000` .

## 3.2 Voltage Frame Formatting

**objective:** Verify correct formatting of a voltage sample.

- **Stimulus:**
  - Assert **`spi_data_valid` ;**
  - Send a frame with:
- Channel ID = `4'h1` ;
- ADC sample = known value.
- **Expected Result:**
  - `v_sample_out` receives the formatted value;
  - `i_sample_out` remains unchanged;
  - `sample_valid` asserted for one cycle.

## 3.3 Current Frame Formatting

**objective:** Verify correct formatting of a current sample.

- **Stimulus:**
  - Assert `spi_data_valid` ;
  - Send a frame with:
    - Channel ID = `4'h2` ;
    - ADC sample = known value.
- **Expected Result:**
  - `i_sample_out` receives the formatted value;
  - `v_sample_out` remains unchanged;
  - `sample_valid` asserted for one cycle.

## 3.4 Zero Extension

**objective:** Verify proper zero-extension from 12 bits to 16 bits.

- **Stimulus:**
  - Send several 12-bit values.
- **Expected Result:**
  - Correct extension (upper four bits must always be zero).

## 3.5 Invalid Channel ID

**objective:** Verify behavior for unsupported channel IDs.

- **Stimulus:**
  - Send samples using Channel IDs different from `4'h1` and `4'h2` .

- **Expected Result:**
  - Neither output is modified;
  - `sample_valid = 1'b0 .`

## 3.6 Minimum ADC Value

**objective:** Verify formatting of the minimum input.

**Stimulus**:

- ADC sample = `12'h000 .`

- **Expected Result:**
  - Formatted output = `16'h0000 .`

## 3.7 Maximum ADC Value

**objective:** Verify formatting of the maximum input.

- **Stimulus:**
  - ADC sample = `12'hFFF .`
- **Expected Result:**
  - Formatted output = `16'h0FFF .`

## 3.8 Consecutive Frames

**objective:** Verify continuous operation.

- **Stimulus:**
  - Send multiple consecutive SPI frames alternating voltage and current channels.
- **Expected Result:**
  - No frame lost;

- Every frame updates the corresponding output;
- Every frame produces one `sample_valid` pulse.

## 3.9 Random Frames

**objective:** Verify overall functionality.

- **Stimulus:**
  - Generate random Channel IDs and random ADC values.
- **Expected Result:**
  - The formatted outputs match the reference model for every valid transaction.

## 3.10 `sample_valid` Timing

**objective:** Verify protocol timing.

- **Stimulus:**
  - Send several valid SPI frames.
- **Expected Result:**
  - `sample_valid` is asserted for exactly one clock cycle;
  - Pulse occurs only after processing reaches the output stage;
  - No false pulses occur.

## 3.11 FSM State Transition

**objective:** Verify correct FSM sequence.

- **Stimulus:**
  - Send one valid SPI transaction.
- **Expected Result:**
  - FSM follows: `STATE_IDLE` -> `STATE_EXTEND_ALIGN` -> `STATE_CH_SEPARATOR` ->

    `STATE_REG_OUTPUT` -> `STATE_IDLE` ;
  - No illegal transitions occur.

## 3.12 No Valid Input

**objective:** Verify idle behavior.

- **Stimulus:**
  - Keep `spi_data_valid = 0` .
- **Expected Result:**
  - FSM remains in `STATE_IDLE` ;
  - Outputs remain unchanged;
  - `sample_valid` stays low.

# Signal Processing Verification Test Plan

## 1. Overview and Scope

This document details the verification plan for the `signal_processing` RTL module. The Design Under Test (DUT) is a signal processing module, its function is to receive voltage and current samples, apply a four-sample moving average filter, and provide filtered outputs with a valid pulse indicating that a new filtered value is available.

**Input Signals:** `clk`, `rst_n`, `v_sample_out`, `i_sample_out`, `sample_valid` **Output Signals:** `v_signal_out`, `i_signal_out`, `signal_valid`

## 2. Verification Strategy

The verification strategy is based on RTL simulation using a self-checking testbench combined with SystemVerilog Assertions (SVA). Functional correctness is verified by comparing DUT outputs against a reference moving-average model implemented inside the testbench.
Assertions are used to verify protocol timing, reset behavior and output validity.

## 3. Test Cases

### 3.1 Reset and Inicialization

**objective:** Ensure that the reset initializes correctly.

- **Stimulus:** Assert and de-assert `rst_n` (active-low).
- **Expected Result:**
  - FSM returs to `STATE_IDLE`;
  - `v_signal_out = 16Th0000`;

- `i_signal_out = 16Th0000`;
- `signal_valid = 1Tb0`;
- `v_sum = 18Th00000`;
- `i_sum = 18Th00000`;
- All positions of `v_reg` and `v_reg` goes to `16Th0000`.

## 3.2 Single Sample Processing

**objective:** Check the first sample.

- **Stimulus:**
  - Assert `sample_valid` (active-high);
  - Assign one sample to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - `v_signal_out = v_sample_out / 4`;
  - `i_signal_out = i_sample_out / 4`.

## 3.3 Pipeline Filling

**objective:** Check the pipeline filling.

- **Stimulus:**
  - Assert `sample_valid` (active-high);
  - Assign four samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - First sample:
    - `v_signal_out = v_sample_1 / 4`;
    - `i_signal_out = i_sample_1 / 4`;
  - Second sample:
    - `v_signal_out = (v_sample_1 + v_sample_2) / 4`;
    - `i_signal_out = (i_sample_1 + i_sample_2) / 4`;
  - Third sample:
    - `v_signal_out = (v_sample_1 + v_sample_2 + v_sample_3) / 4`;

        - `i_signal_out = (i_sample_1 + i_sample_2 + i_sample_3) / 4;`
    - Fourth sample:
        - `v_signal_out = (v_sample_1 + v_sample_2 + v_sample_3 + v_sample_4) / 4;`
        - `i_signal_out = (i_sample_1 + i_sample_2 + i_sample_3 + i_sample_4) / 4`.

### 3.4 Sliding Window Verification

**objective:** Check the disposal of the oldest sample.

- **Stimulus:**
    - Assign five samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
    - Discard the oldest sample, so that the fifth sample is:
        - `v_signal_out = (v_sample_2 + v_sample_3 + v_sample_4 + v_sample_5) / 4;`
        - `i_signal_out = (i_sample_2 + i_sample_3 + i_sample_4 + i_sample_5) / 4`.

### 3.5 Constant Inputs

**objective:** Check stability.

- **Stimulus:**
    - Assign four equal samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
    - The final result must be equal to the input:
        - `v_signal_out = v_sample_out;`
        - `i_signal_out = i_sample_out`.

### 3.6 Increasing Ramp

**objective:** Check the rising average.

- **Stimulus:**
  - Assign rising samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must increase and correspond to a known result.

## 3.7 Decreasing Ramp

**objective:** Check the declining average.

- **Stimulus:**
  - Assign declining samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must decrease and correspond to a known result.

## 3.8 Maximum Values

**objective:** Check overflow.

- **Stimulus:**
  - Assign maximum values to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must correspond to a maximum known value;
  - The internal sum must not overflow.

## 3.9 Minimum Values

**objective:** Check zeros.

- **Stimulus:**
  - Assign zero to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must be zero.

## 3.10 Random Samples

**objective:** Check reliability and functionality.

- **Stimulus:**
  - Assign random values to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The result must be equal to the one calculated automatically.

## 3.11 `signal_valid` Timing

**objective:** Ensure protocol.

- **Stimulus:**
  - Assign values to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - Single pulse;
  - Occurs after processing;
  - Never occurs spontaneously.

# MPPT Core Test Plan

## Assumptions (Input Constraints)

### `valid_signal_is_single_cycle_pulse`

**Description**: Legal transactions present `valid_signal` as a one-cycle pulse so the DUT's rising-edge detector starts exactly one MPPT evaluation cycle. The RTL detects the valid pulse as `valid_pulse = valid_signal & ~valid_d`, where `valid_d` is registered on `clk`.
**Signals**: `valid_signal` **Notes**:

```
$rose(valid_signal) |-> ##1 !valid_signal;
```

### `sample_inputs_stable_during_evaluation`

**Description**: A legal MPPT sample holds `V` and `I` stable long enough for the controller sequence to compare the current power and voltage against the stored previous sample. The RTL computes `P_new = V * I` combinationally and reuses it throughout the FSM evaluation sequence. **Signals**: `V`, `I`, `valid_signal` **Notes**: After a `valid_signal` pulse, hold `V` and `I` stable for the bounded evaluation window (approximately four cycles). This is an input-only constraint.

## Interface Assertions

## reset_forces_idle_outputs

**Type**: Assertion **Targets**: `Dc_out`, `valid_end_cycle` **Description**: Asserting active-high `rst` asynchronously clears the edge detector and returns the FSM to the `IDLE` state. Since `Dc_out` defaults to `2Tb10` outside the action states and `valid_end_cycle` is asserted only during the `END` state, reset should force the observable idle output pattern. **Stimulus / Antecedent**:

```
rst == 1Tb1
```

**Expected Consequent**:

```
Dc_out == 2Tb10
  valid_end_cycle == 1Tb0
```

**Timing**: Same cycle (asynchronous reset) and remains true while reset is asserted. **Auxiliary Logic**: None. **Success Criteria**: Reset always presents the idle/no-action output state and suppresses completion.

---

## no_completion_without_accepted_valid

**Type**: Assertion **Targets**: `valid_end_cycle` **Description**: `valid_end_cycle` shall not assert unless a previously accepted `valid_signal` rising edge has started an MPPT evaluation. The DUT leaves `IDLE` only on `valid_pulse`. **Stimulus / Antecedent**: No pending transaction exists, and no accepted `valid_signal` rising edge has occurred during the bounded observation window. **Expected Consequent**:

```
valid_end_cycle == 1Tb0
```

**Timing**: Continuous outside the one-to-four-cycle transaction window. **Auxiliary Logic**: 1-bit pending flag and a 3-bit transaction-age counter

started by `$rose(valid_signal)` and cleared by `valid_end_cycle`. **Success Criteria**: The completion flag never asserts spuriously during idle operation.

---

## valid_end_single_cycle_pulse

**Type**: Assertion **Targets**: `valid_end_cycle` **Description**: Completion is encoded by the one-cycle `END` state because `valid_end_cycle = (state == END)`. The FSM immediately transitions from `END` back to `IDLE`. **Stimulus / Antecedent**:

```
valid_end_cycle == 1Tb1
```

**Expected Consequent**:

```
##1 !valid_end_cycle
```

**Timing**: One cycle. **Auxiliary Logic**: None. **Success Criteria**: Completion is a one-cycle pulse rather than a level.

## equal_power_completes_without_action

**Type**: Assertion **Targets**: `valid_end_cycle`, `Dc_out` **Description**: When a newly accepted sample has the same computed power as the stored previous power, the FSM transitions directly from `IDLE` to `END` without changing the duty command from its default value. This matches the Perturb and Observe (P&O) algorithm, where no perturbation is required when the measured power does not change. **Stimulus / Antecedent**: Accepted `valid_signal` pulse when the auxiliary shadow register satisfies `prev_power == V * I`. **Expected Consequent**: `valid_end_cycle` asserts during the bounded completion cycle and `Dc_out == 2Tb10`. **Timing**: One cycle after the accepted sample, using the monitor's registered transaction-age

counter. **Auxiliary Logic**: Shadow `prev_power`, updated when `valid_end_cycle` completes; pending flag and transaction-age counter. **Success Criteria**: Equal-power samples complete without issuing action codes `2Tb00` or `2Tb01`.

---

## same_direction_reference_increase_code

**Type**: Assertion **Targets**: `Dc_out` **Description**: When power and voltage change in the same direction, the controller perturbs the reference voltage in the same direction. The RTL maps both `(curr_power > prev_power && V > prev_V)` and `(curr_power < prev_power && V < prev_V)` to the `V_ref_G` state, where `Dc_out == 2Tb00`. **Stimulus / Antecedent**: Accepted sample satisfying `(curr_power > prev_power && V > prev_V)` or `(curr_power < prev_power && V < prev_V)`. **Expected Consequent**: `Dc_out == 2Tb00` during the bounded action cycle before completion. **Timing**: Second cycle after the accepted sample; completion follows one cycle later. **Auxiliary Logic**: Shadow `prev_power`, shadow `prev_V`, and transaction-age counter. **Success Criteria**: Same-direction power and voltage changes always produce the reference-increase command.

---

## same_direction_sequence_completes

**Type**: Assertion **Targets**: `valid_end_cycle` **Description**: After taking the same-direction branch, the FSM must progress through the action state and eventually reach `END`, asserting `valid_end_cycle`. **Stimulus / Antecedent**: Same antecedent as `same_direction_reference_increase_code`. **Expected Consequent**: `valid_end_cycle == 1Tb1`. **Timing**: Third cycle after the accepted sample. **Auxiliary Logic**: Reuse the shadow registers and transaction-age counter. **Success Criteria**: The same-direction branch cannot deadlock before completion.

---

## opposite_direction_reference_decrease_c

**Type**: Assertion **Targets**: `Dc_out` **Description**: When power and voltage change in opposite directions, the controller perturbs the reference voltage in the opposite direction. The RTL maps `(curr_power > prev_power && V < prev_V)` and `(curr_power < prev_power && V > prev_V)` to the `V_ref_L` state, where `Dc_out == 2Tb01`. **Stimulus / Antecedent**: Accepted sample satisfying `(curr_power > prev_power && V < prev_V)` or `(curr_power < prev_power && V > prev_V)`. **Expected Consequent**: `Dc_out == 2Tb01` during the bounded action cycle before completion. **Timing**: Second cycle after the accepted sample; completion follows one cycle later. **Auxiliary Logic**: Shadow `prev_power`, shadow `prev_V`, and transaction-age counter. **Success Criteria**: Opposite-direction power and voltage changes always produce the reference-decrease command.

## opposite_direction_sequence_completes

**Type**: Assertion **Targets**: `valid_end_cycle` **Description**: After taking the opposite-direction branch, the FSM must progress through the action state and eventually reach `END`, asserting `valid_end_cycle`.
**Stimulus / Antecedent**: Same antecedent as `opposite_direction_reference_decrease_code`. **Expected Consequent**: `valid_end_cycle == 1Tb1`. **Timing**: Third cycle after the accepted sample. **Auxiliary Logic**: Reuse the shadow registers and transaction-age counter. **Success Criteria**: The opposite-direction branch cannot deadlock before completion.

## ambiguous_delta_keeps_default_command

**Type**: Assertion **Targets**: `Dc_out` **Description**: If power changes but voltage remains unchanged relative to the stored previous voltage, neither directional P&O branch condition is satisfied. The FSM therefore transitions from `P_DIFF` directly to `END` without entering either `V_ref_L` or `V_ref_G`, leaving the duty command at its default value. **Stimulus / Antecedent**: Accepted `valid_signal` pulse where `curr_power != prev_power` and `V == prev_V`. **Expected Consequent**: `Dc_out == 2Tb10` throughout the bounded transaction; no action code is emitted.
**Timing**: Bounded over the two-cycle `P_DIFF` → `END` path. **Auxiliary Logic**: Shadow `prev_power`, shadow `prev_V`, pending flag, and transaction-age counter. **Success Criteria**: An ambiguous voltage delta never generates a false reference increase or decrease command.

---

## `ambiguous_delta_completes`

**Type**: Assertion **Targets**: `valid_end_cycle` **Description**: Even when the voltage comparison cannot select either directional branch, the FSM must still terminate by taking the default `P_DIFF` → `END` path and asserting `valid_end_cycle`. **Stimulus / Antecedent**: Same antecedent as `ambiguous_delta_keeps_default_command`. **Expected Consequent**: `valid_end_cycle == 1Tb1`. **Timing**: Second cycle after the accepted sample. **Auxiliary Logic**: Reuse the shadow registers and transaction-age counter from `ambiguous_delta_keeps_default_command`. **Success Criteria**: The controller always completes the transaction, even when the voltage comparison is ambiguous.

---

## `back_to_back_samples_use_previous_compl`

**Type**: Assertion **Targets**: `Dc_out`, `valid_end_cycle` **Description**: A sample accepted after a completed transaction shall be compared against the voltage and power captured at the previous completed transaction. The RTL updates `prev_V` and `prev_power` only during the

`END` state, so each new transaction must use the most recently completed sample as its comparison baseline. **Stimulus / Antecedent**: Drive one legal transaction to completion. Then drive a second legal `valid_signal` pulse with `V` and `I` selected so that the relationship between the second sample and the first completed sample produces a known directional branch. **Expected Consequent**: The second transaction emits the command corresponding to the relationship with the first completed sample (`Dc_out == 2Tb00` for same-direction deltas or `Dc_out == 2Tb01` for opposite-direction deltas), followed by `valid_end_cycle == 1Tb1`. **Timing**: During the second transaction, the action code is produced on the second cycle after the accepted sample, and `valid_end_cycle` asserts on the third cycle. **Auxiliary Logic**: Shadow `prev_power` and `prev_V` updated only when `valid_end_cycle` is observed, a two-transaction phase flag, and a transaction-age counter. **Success Criteria**: Consecutive transactions always use the previous completed sample as the comparison baseline rather than reset values or the current sample itself.